

DBMS PROJECT REPORT

Order Management System

Student Name	Balaji MJ
Register Number	TU-6253210211004
Week	Week 4
Subject	Database Management System
Task	Order Management System

Task Description

Design and implement an Order Management Database using MySQL. The system manages customers, products, orders, and order details, supports data insertion and updates, and retrieves customer order history through relational joins.

1. Database and Table Design

The database is named **OrderManagementDB**. Four related tables are created: Customer, Product, Orders, and Order_Details.

```
CREATE DATABASE OrderManagementDB;
USE OrderManagementDB;

CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(15)
);

CREATE TABLE Product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2)
);
```

The Customer table stores customer identity and contact information, while Product stores product names and prices.

2. Orders and Order Details

The Orders table records the customer associated with an order, order date, and total amount. The Order_Details table records the individual products, quantities, and unit prices belonging to each order.

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE,  
    total_amount DECIMAL(10,2),  
    FOREIGN KEY (customer_id)  
        REFERENCES Customer(customer_id)  
);
```

```
CREATE TABLE Order_Details (  
    order_detail_id INT PRIMARY KEY,  
    order_id INT,  
    product_id INT,  
    quantity INT,  
    unit_price DECIMAL(10,2),  
    FOREIGN KEY (order_id)  
        REFERENCES Orders(order_id),  
    FOREIGN KEY (product_id)  
        REFERENCES Product(product_id)  
);
```

Foreign keys maintain relationships between customers and orders, and between orders, products, and order details.

3. Customer and Product Data

The supplied script inserts five customers and six products. Example products include Laptop, Mobile Phone, Keyboard, Mouse, Headphones, and Monitor.

```
INSERT INTO Customer
(customer_id, customer_name, email, phone)
VALUES
(1, 'Rahul Sharma', 'rahul@gmail.com', '9876543210'),
(2, 'Priya Singh', 'priya@gmail.com', '9876543211'),
(3, 'Aman Verma', 'aman@gmail.com', '9876543212'),
(4, 'Sneha Yadav', 'sneha@gmail.com', '9876543213'),
(5, 'Rohit Kumar', 'rohit@gmail.com', '9876543214');
```

```
INSERT INTO Product
(product_id, product_name, price)
VALUES
(101, 'Laptop', 55000.00),
(102, 'Mobile Phone', 25000.00),
(103, 'Keyboard', 1500.00),
(104, 'Mouse', 800.00),
(105, 'Headphones', 2000.00),
(106, 'Monitor', 12000.00);
```

4. Order and Order Detail Data

Five sample orders are inserted for customers 1 through 4. Order details associate each order with one or more products and record quantity and unit price.

```
INSERT INTO Orders
(order_id, customer_id, order_date, total_amount)
VALUES
(1001, 1, '2026-08-10', 57300.00),
(1002, 2, '2026-08-11', 25000.00),
(1003, 1, '2026-08-12', 2300.00),
(1004, 3, '2026-08-13', 14000.00),
(1005, 4, '2026-08-14', 2000.00);

INSERT INTO Order_Details
(order_detail_id, order_id, product_id, quantity, unit_price)
VALUES
(1, 1001, 101, 1, 55000.00),
(2, 1001, 104, 1, 800.00),
(3, 1001, 103, 1, 1500.00),
(4, 1002, 102, 1, 25000.00),
(5, 1003, 104, 1, 800.00),
(6, 1003, 103, 1, 1500.00),
(7, 1004, 106, 1, 12000.00),
(8, 1004, 103, 1, 1500.00),
(9, 1004, 104, 1, 500.00),
(10, 1005, 105, 1, 2000.00);
```

5. Display Operations

The script retrieves the complete Orders and Order_Details tables using SELECT statements.

```
SELECT * FROM Orders;  
  
SELECT * FROM Order_Details;
```

These queries are useful for verifying the inserted order records and individual line items.

6. Update Operations

Two update operations are included. The first changes the total amount of order 1001. The second changes the quantity of order detail 2 from 1 to 2.

```
UPDATE Orders  
SET total_amount = 58000.00  
WHERE order_id = 1001;  
  
UPDATE Order_Details  
SET quantity = 2  
WHERE order_detail_id = 2;
```

7. Customer Order History

The main reporting query combines Customer, Orders, Order_Details, and Product using INNER JOIN operations. It displays customer name, order ID, order date, product name, quantity, unit price, and order total.

```
SELECT
    Customer.customer_name,
    Orders.order_id,
    Orders.order_date,
    Product.product_name,
    Order_Details.quantity,
    Order_Details.unit_price,
    Orders.total_amount
FROM Customer
JOIN Orders
    ON Customer.customer_id = Orders.customer_id
JOIN Order_Details
    ON Orders.order_id = Order_Details.order_id
JOIN Product
    ON Order_Details.product_id = Product.product_id
ORDER BY Orders.order_id;
```

The supplied script ends with an incomplete ORDER BY expression ('ORDER BY Orders.orde'). For this report, the intended completion is shown as ORDER BY Orders.order_id so the query is syntactically complete.

8. Database Relationships

Table	Primary Key	Related Key	Purpose
Customer	customer_id	Referenced by Orders	Stores customer details
Product	product_id	Referenced by Order_Details	Stores product details
Orders	order_id	customer_id → Customer	Stores order information
Order_Details	order_detail_id	order_id → Orders; product_id → Product	Stores order line items

The relational structure supports a one-to-many relationship between Customer and Orders, and between Orders and Order_Details. Products can appear in multiple order-detail records.

9. Expected Functional Output

Operation	Expected Result
Create database	OrderManagementDB is created and selected
Create tables	Four related tables are created
Insert data	Customer, product, order, and detail records are stored
Display orders	All order records are retrieved
Display order details	All line-item records are retrieved
Update order	Order 1001 total becomes 58000.00
Update quantity	Order detail 2 quantity becomes 2
Customer history	Joined customer, order, product, and detail information is displayed

10. Conclusion

The Week-4 Order Management System demonstrates a relational DBMS implementation using MySQL. It creates linked customer, product, order, and order-detail tables, inserts sample records, performs updates, and produces a customer order history through SQL joins.

Final Deliverable: A functional MySQL database script and relational order-history query for OrderManagementDB.